

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Обработка изображений в интеллектуальных
системах»

Тема: «Конструирование моделей на базе предобученных
нейронных сетей»

Выполнил:

Студент 4 курса
Группы ИИ-26
Ковальчук А.И.

Проверила:

Андренко К.В.

Цель: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Общее задание:

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Вариант 7

В-т	Выборка	Оптимизатор	Предобученная архитектура
7	Fashion-MNIS T	Adam	ResNet34

Программная реализация:

```
def build_resnet34(num_classes=NUM_CLASSES, pretrained=True,
freeze_backbone=FREEZE_BACKBONE):

    weights = ResNet34_Weights.DEFAULT if pretrained else None

    model = resnet34(weights=weights)
```

```

original_conv1 = model.conv1

model.conv1 = nn.Conv2d(

    in_channels=1,

    out_channels=original_conv1.out_channels,

    kernel_size=original_conv1.kernel_size,

    stride=original_conv1.stride,

    padding=original_conv1.padding,

    bias=False,

)

with torch.no_grad():

    if pretrained:

        model.conv1.weight.copy_(original_conv1.weight.mean(dim=1,
keepdim=True))

    else:

        nn.init.kaiming_normal_(model.conv1.weight, mode="fan_out",
nonlinearity="relu")

model.fc = nn.Linear(model.fc.in_features, num_classes)

if freeze_backbone:

    for name, param in model.named_parameters():

        param.requires_grad = name.startswith("fc") or
name.startswith("conv1")

return model

```

```

with torch.no_grad():

    _check_model = build_resnet34(pretrained=False).eval()

    _check = _check_model(torch.randn(2, 1, INPUT_SIZE, INPUT_SIZE))

print("Проверка forward:", tuple(_check.shape))

print("Параметров:", f"{count_parameters(_check_model):,}")

del _check_model

@torch.no_grad()

def run_eval_epoch(model, dataloader, criterion, device):

    model.eval()

    running_loss = 0.0

    correct = 0

    total = 0

    for inputs, labels in dataloader:

        inputs = inputs.to(device, non_blocking=True)

        labels = labels.to(device, non_blocking=True)

        with autocast_ctx(device):

            outputs = model(inputs)

            loss = criterion(outputs, labels)

            running_loss += loss.item() * inputs.size(0)

            correct += (outputs.argmax(dim=1) == labels).sum().item()

            total += labels.size(0)

    return running_loss / total, correct / total

```

```

def train_model(model, train_loader, val_loader, criterion, optimizer,
device, num_epochs, writer=None, scheduler=None):

    history = {"train_loss": [], "val_loss": [], "train_acc": [],
"val_acc": []}

    best_val_acc = 0.0

    best_state = None

    scaler = make_grad_scaler(device)

    for epoch in range(num_epochs):

        model.train()

        running_loss = 0.0

        correct = 0

        total = 0

        for inputs, labels in train_loader:

            inputs = inputs.to(device, non_blocking=True)

            labels = labels.to(device, non_blocking=True)

            optimizer.zero_grad(set_to_none=True)

            with autocast_ctx(device):

                outputs = model(inputs)

                loss = criterion(outputs, labels)

            scaler.scale(loss).backward()

            if GRAD_CLIP_NORM:

                scaler.unscale_(optimizer)

                nn.utils.clip_grad_norm_(model.parameters(),
GRAD_CLIP_NORM)

```

```
        scaler.step(optimizer)

        scaler.update()

        running_loss += loss.item() * inputs.size(0)

        correct += (outputs.argmax(dim=1) == labels).sum().item()

        total += labels.size(0)

    train_loss = running_loss / total

    train_acc = correct / total

    val_loss, val_acc = run_eval_epoch(model, val_loader, criterion,
device)

    if scheduler is not None:

        scheduler.step()

    history["train_loss"].append(train_loss)

    history["val_loss"].append(val_loss)

    history["train_acc"].append(train_acc)

    history["val_acc"].append(val_acc)

    if writer is not None:

        writer.add_scalar("Loss/train", train_loss, epoch + 1)

        writer.add_scalar("Loss/val", val_loss, epoch + 1)

        writer.add_scalar("Accuracy/train", train_acc, epoch + 1)

        writer.add_scalar("Accuracy/val", val_acc, epoch + 1)

    if val_acc > best_val_acc:
```

```

        best_val_acc = val_acc

        best_state = {k: v.detach().cpu().clone() for k, v in
model.state_dict().items()}

        current_lrs = ", ".join(f"{group['lr']:.2e}" for group in
optimizer.param_groups)

        print(

            f"Epoch {epoch + 1}/{num_epochs} - "

            f"train_loss: {train_loss:.4f}, train_acc: {train_acc:.4f},
"

            f"val_loss: {val_loss:.4f}, val_acc: {val_acc:.4f}, lr:
[{current_lrs}]"

        )

        if best_state is not None:

            model.load_state_dict(best_state)

        print(f"Лучшая val_acc: {best_val_acc:.4f}")

        return history

```

Результаты обучения

Сеть обучалась со следующими установленными гиперпараметрами:

```

NUM_CLASSES = len(CLASS_NAMES)

IMAGE_SIZE = 28

INPUT_SIZE = 224

FASHION_MEAN = 0.2860

FASHION_STD = 0.3530

```

```
BATCH_SIZE = 64

NUM_EPOCHS = 15

LEARNING_RATE = 1e-4

HEAD_LEARNING_RATE = 1e-3

WEIGHT_DECAY = 1e-4

LABEL_SMOOTHING = 0.05

ADAM_BETAS = (0.9, 0.999)

ETA_MIN = 1e-6

WARMUP_EPOCHS = 2

GRAD_CLIP_NORM = 1.0

VAL_SIZE = 0.1

SEED = 42

NUM_WORKERS = 2

CPU_THREADS = min(8, os.cpu_count() or 8)
```

Прогресс по эпохам:

Epoch 1/15 - train_loss: 1.9873, train_acc: 0.3717, val_loss: 1.9719, val_acc: 0.3502, lr: [5.50e-05, 5.50e-04]

Epoch 2/15 - train_loss: 0.9015, train_acc: 0.7718, val_loss: 0.8206, val_acc: 0.7679, lr: [1.00e-04, 1.00e-03]

Epoch 3/15 - train_loss: 0.6860, train_acc: 0.8438, val_loss: 0.8285, val_acc: 0.7848, lr: [9.86e-05, 9.85e-04]

Epoch 4/15 - train_loss: 0.6153, train_acc: 0.8802, val_loss: 0.7182, val_acc: 0.8270, lr: [9.43e-05, 9.43e-04]

Epoch 5/15 - train_loss: 0.5336, train_acc: 0.9034, val_loss: 0.6700, val_acc: 0.8523, lr: [8.76e-05, 8.74e-04]

Epoch 6/15 - train_loss: 0.4798, train_acc: 0.9318, val_loss: 0.5907, val_acc: 0.8776, lr: [7.86e-05, 7.84e-04]

Epoch 7/15 - train_loss: 0.4375, train_acc: 0.9446, val_loss: 0.6775, val_acc: 0.8481, lr: [6.81e-05, 6.78e-04]

Epoch 8/15 - train_loss: 0.4114, train_acc: 0.9588, val_loss: 0.5538, val_acc: 0.9072, lr: [5.65e-05, 5.61e-04]

Epoch 9/15 - train_loss: 0.3760, train_acc: 0.9740, val_loss: 0.5271, val_acc: 0.9325, lr: [4.45e-05, 4.40e-04]

Epoch 10/15 - train_loss: 0.3442, train_acc: 0.9863, val_loss: 0.5346, val_acc: 0.9114, lr: [3.29e-05, 3.23e-04]

Epoch 11/15 - train_loss: 0.3345, train_acc: 0.9882, val_loss: 0.5170, val_acc: 0.9241, lr: [2.24e-05, 2.17e-04]

Epoch 12/15 - train_loss: 0.3174, train_acc: 0.9953, val_loss: 0.4922, val_acc: 0.9283, lr: [1.34e-05, 1.27e-04]

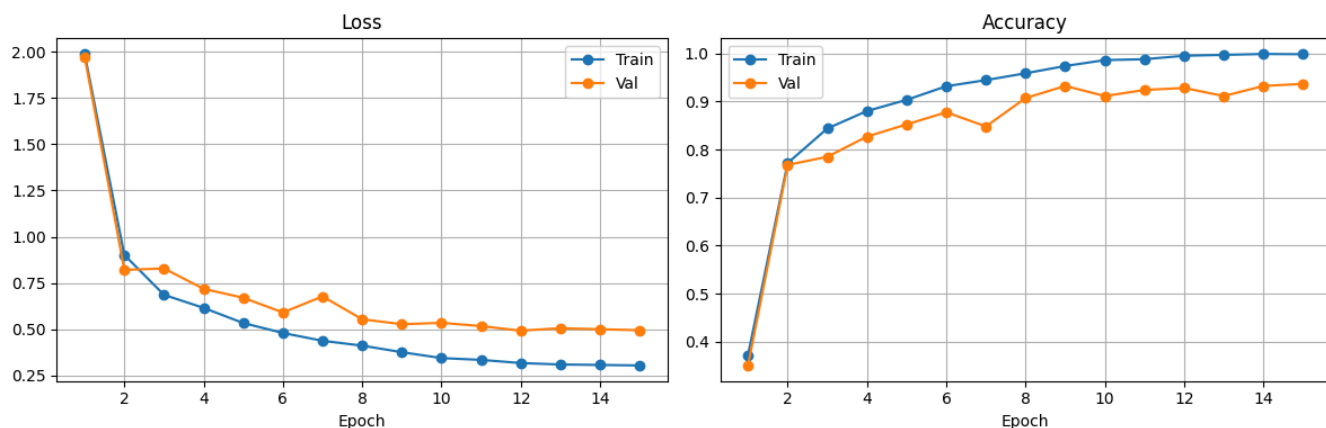
Epoch 13/15 - train_loss: 0.3096, train_acc: 0.9972, val_loss: 0.5058, val_acc: 0.9114, lr: [6.67e-06, 5.82e-05]

Epoch 14/15 - train_loss: 0.3071, train_acc: 0.9991, val_loss: 0.5001, val_acc: 0.9325, lr: [2.44e-06, 1.55e-05]

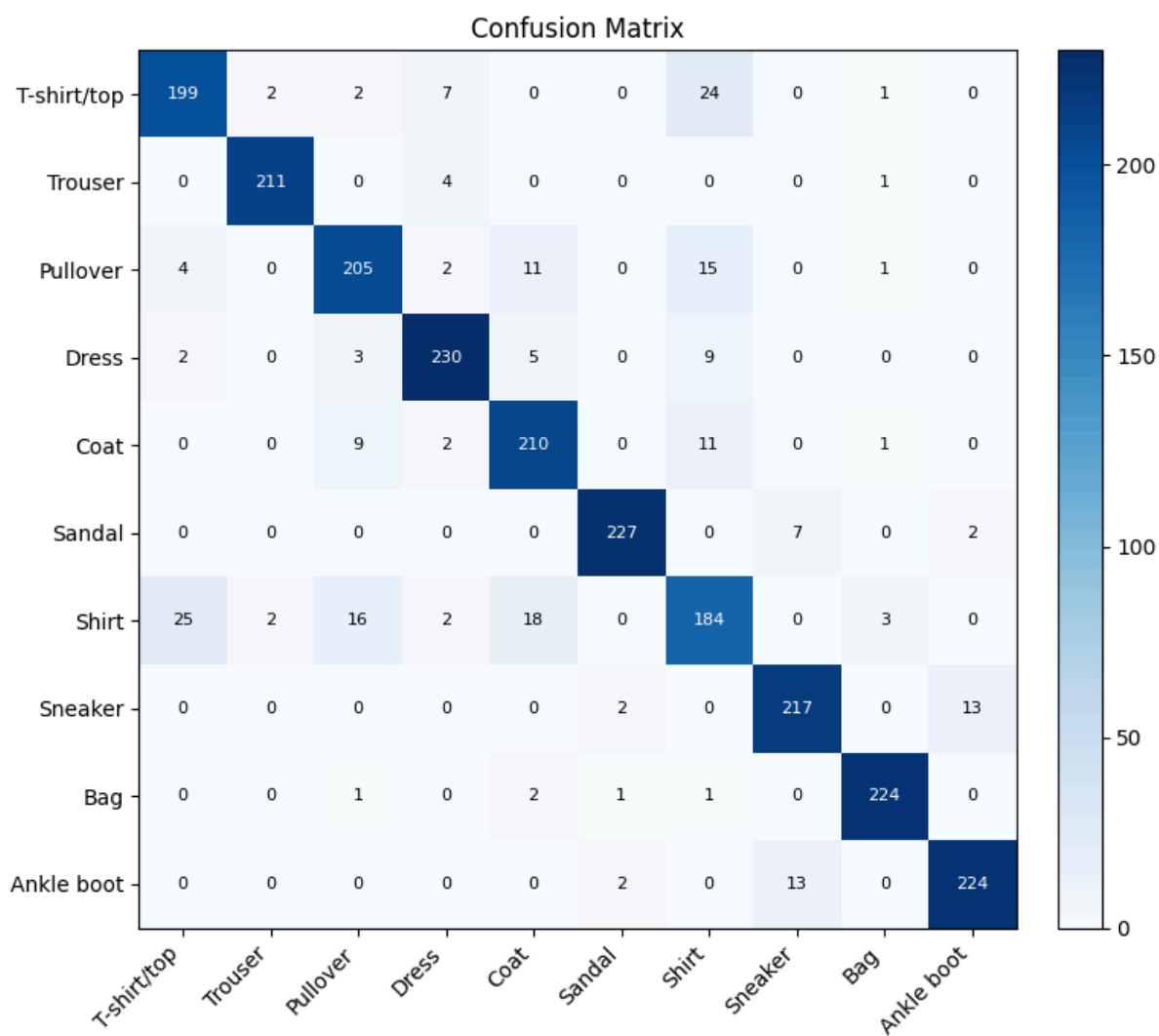
Epoch 15/15 - train_loss: 0.3044, train_acc: 0.9986, val_loss: 0.4947, val_acc: 0.9367, lr: [1.00e-06, 1.00e-06]

Лучшая val_acc: 0.9367

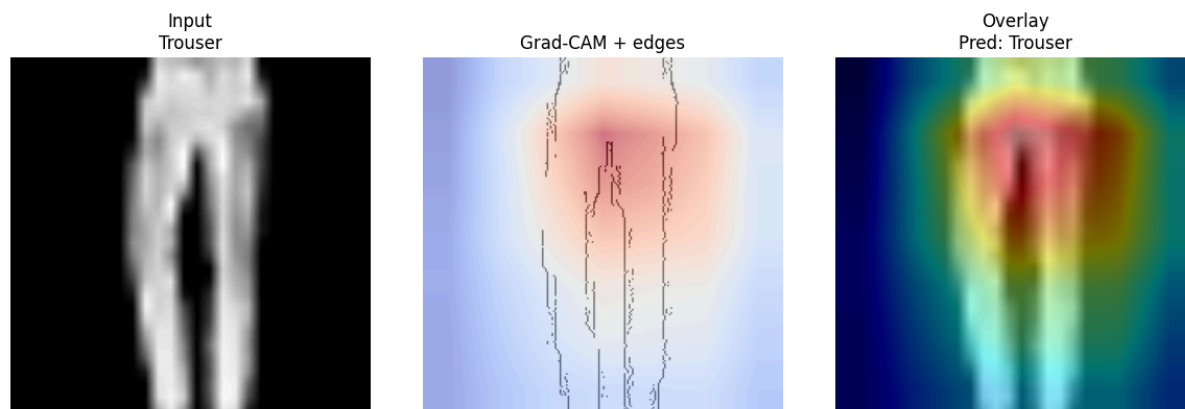
Графики изменения функции потерь и значения точности на обучающем и валидационных наборах данных:



Матрица ошибки:



Тепловая карта активаций выходного свёрточного слоя:



Вывод. Использование предобученных моделей позволяет существенно ускорить обучение и повысить репрезентативную способность сети в рамках интересующей нас задачи. Это достигается за счёт переиспользования полезных закономерностей, которые веса модели усвоили на близкой к нашей задаче.